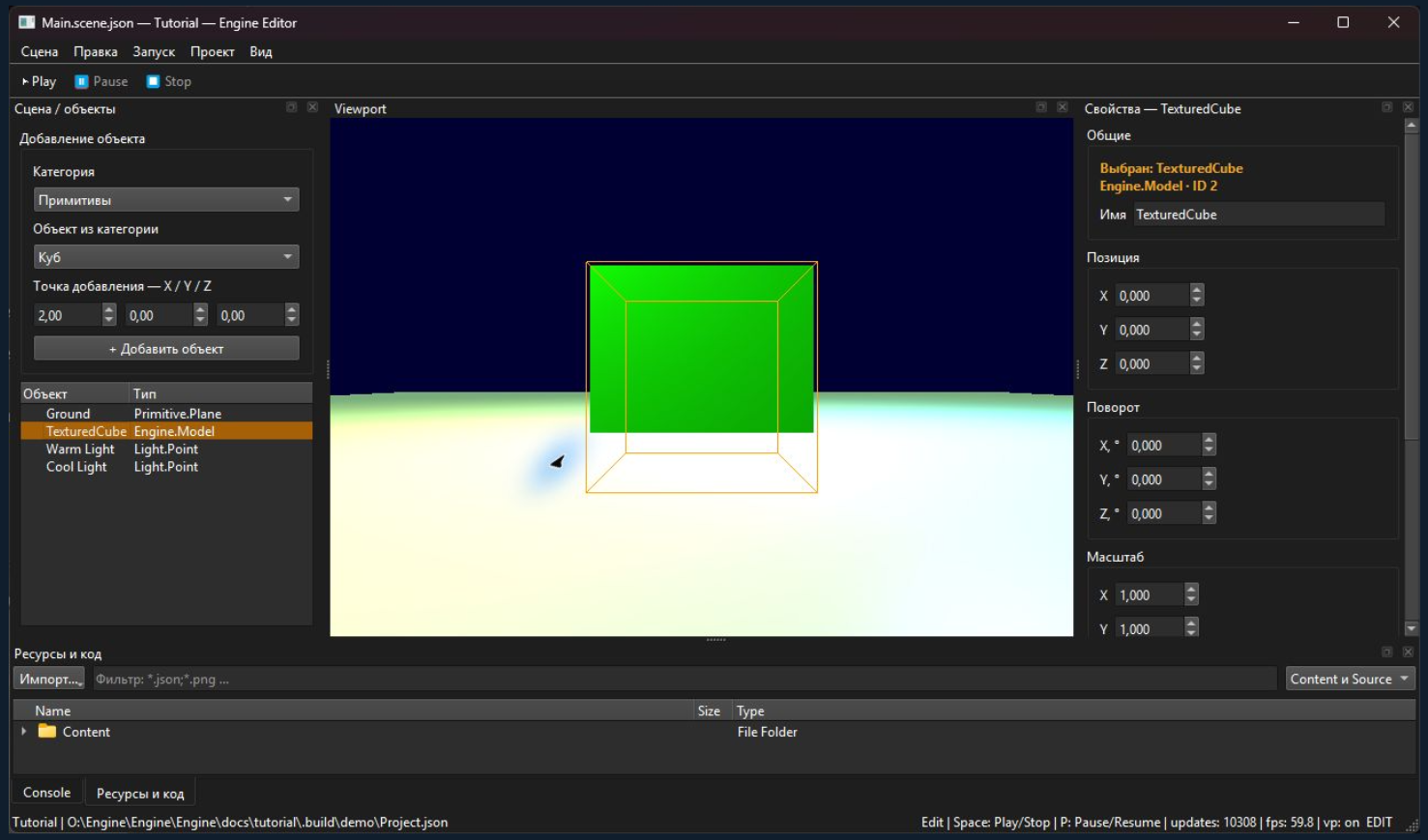
PainEngine Практический туториал

Редактор и рабочие примеры C++



2026.10.28.2 / 30.10.2026

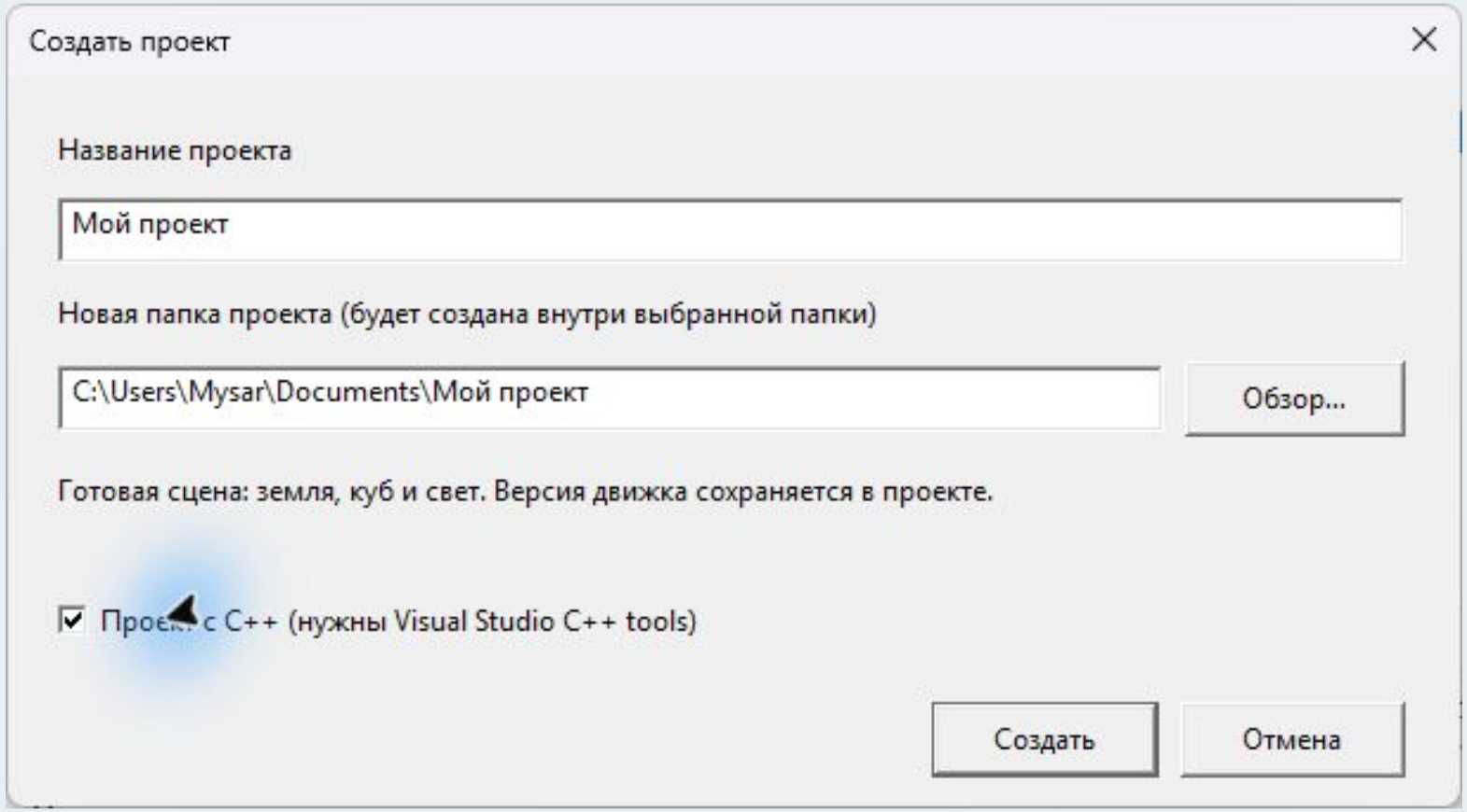
Первый проект в лаунчере

Создать проект

Выберите папку и название. C++ можно включить сразу или использовать шаблон без кода.

Редактировать

Выберите созданный проект в списке лаунчера и откройте редактор.



Флажок C++ создаёт исходники, включая RotatingCube. Нужны Visual Studio C++ tools и Windows SDK.

Окно редактора

Слева

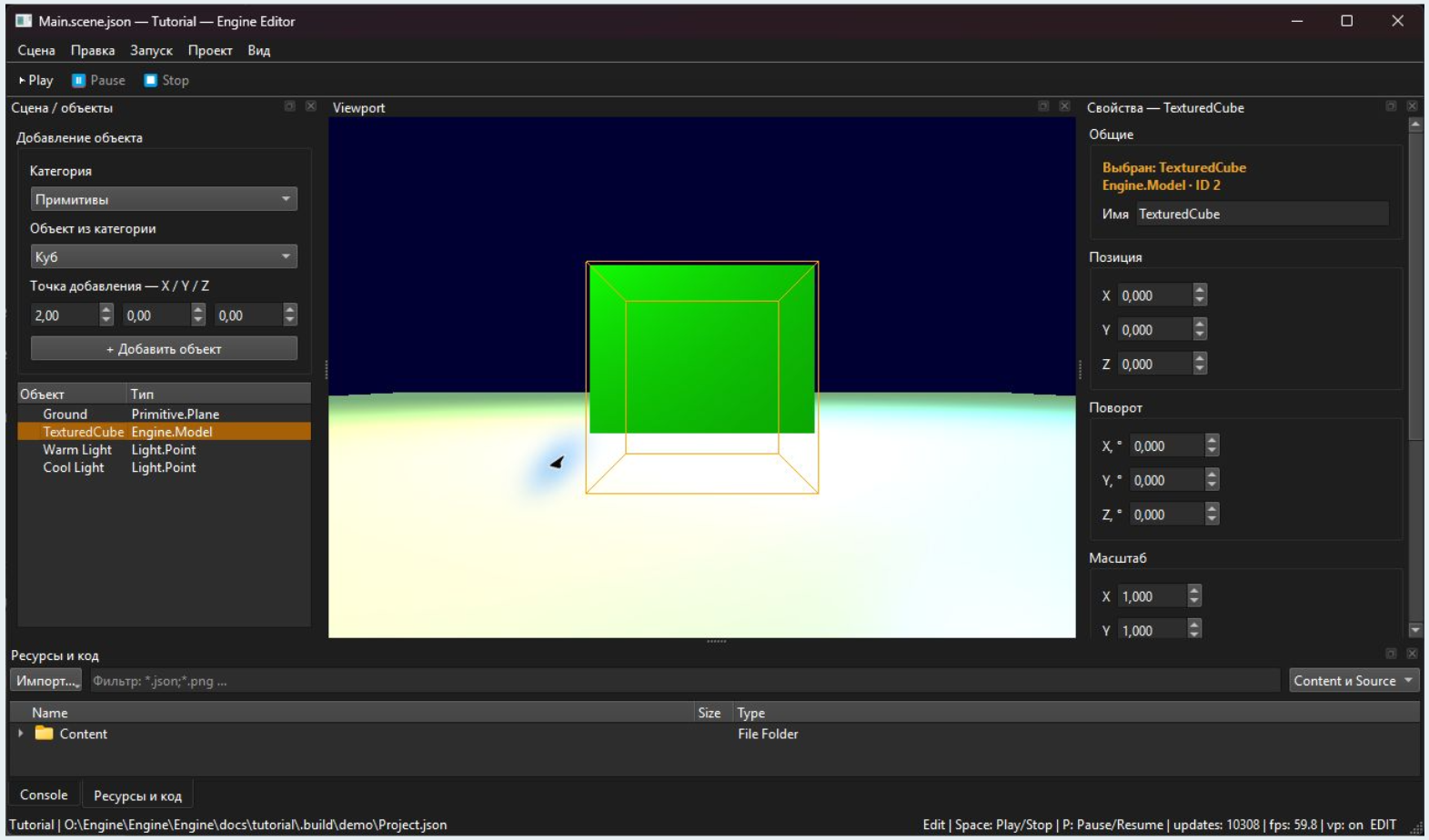
Категория, тип объекта и иерархия сцены.

Справа

Трансформация, модель, текстура и материал объекта.

Снизу

Ресурсы и код; рядом вкладка Console.



Камера и выделение в Edit

Камера

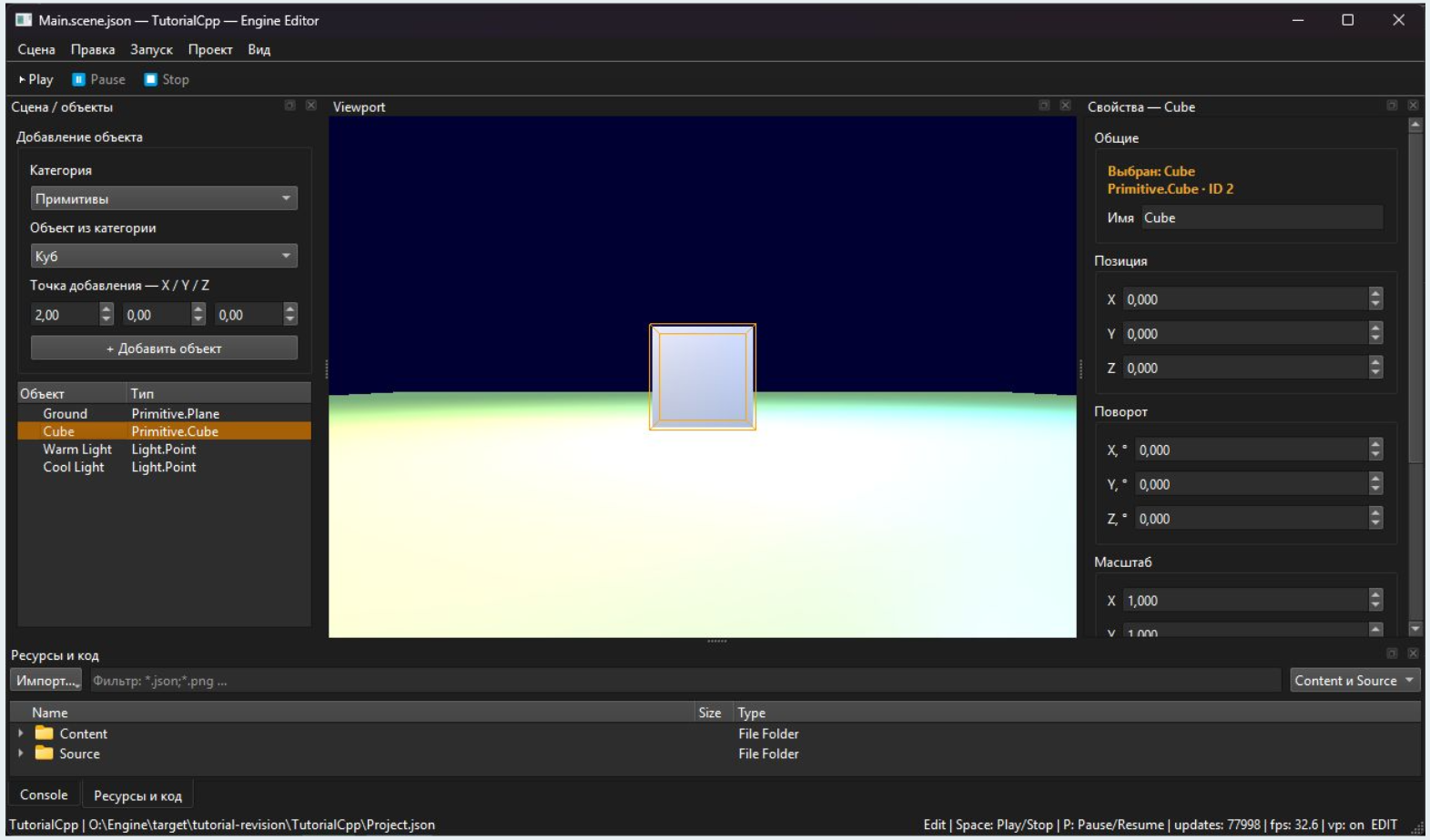
WASD; E / Q вверх и вниз. ПКМ + мышь для обзора, Shift для ускорения.

Выделение

ЛКМ — выбор; Tab — перебор; F — фокус; Esc — снять выбор.

Перемещение

Стрелки и Page Up / Page Down. Точные значения вводятся в свойствах.



Сначала щёлкните viewport. При вводе в текстовое поле горячие клавиши сцены не действуют.

Импорт модели и текстуры

Импорт

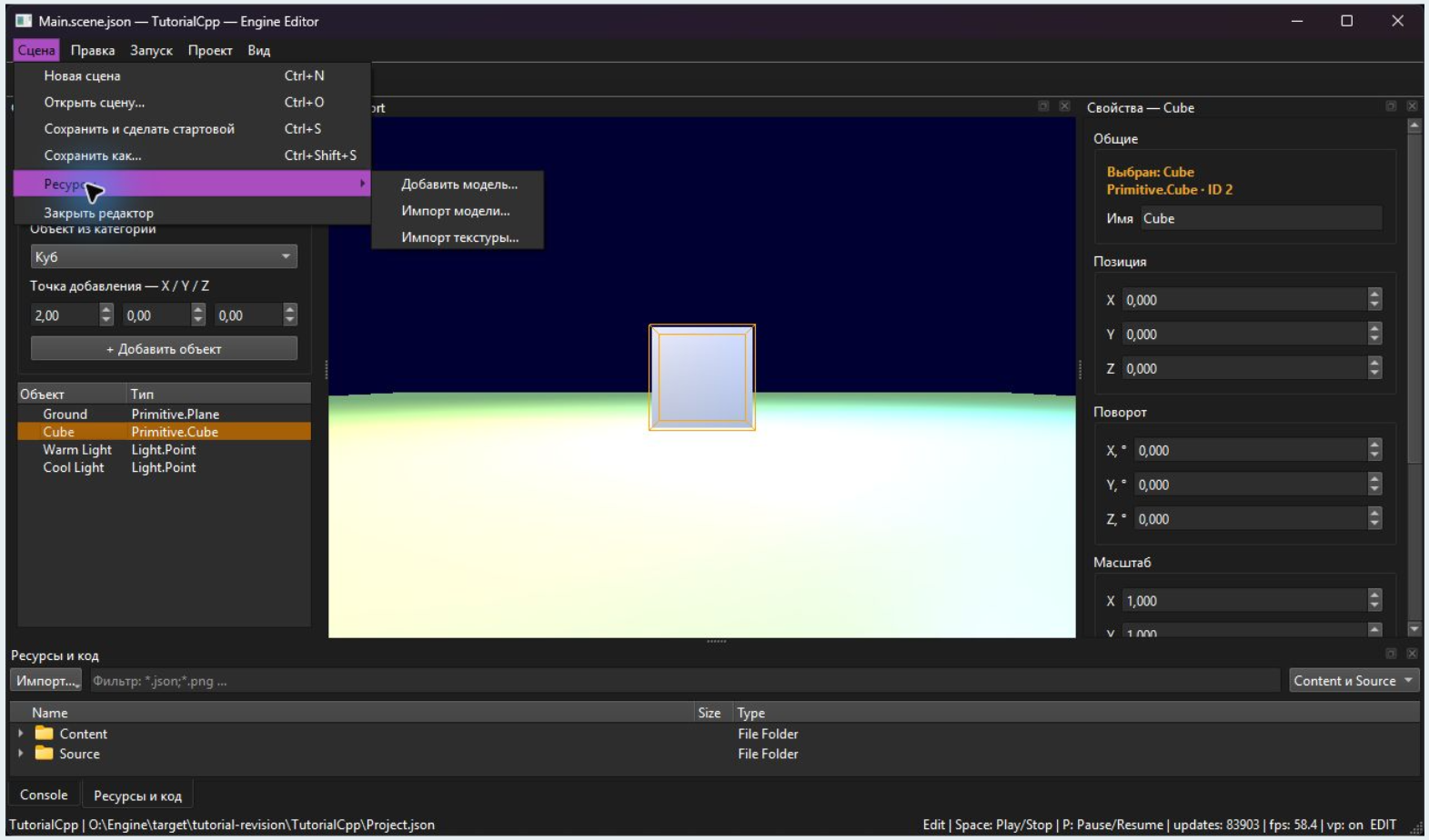
Сцена → Ресурсы → Импорт модели… или Импорт текстуры…

Добавление

Дважды щёлкните модель в Content/Models, чтобы добавить её в сцену.

Назначение

Для отдельной текстуры: выберите объект и нажмите «Выбрать текстуру…».



Модели: GLB, glTF, OBJ, FBX, BLEND. Изображения: PNG, JPG, BMP, TGA.

FBX / BLEND: модель сразу с материалом

Что создаётся

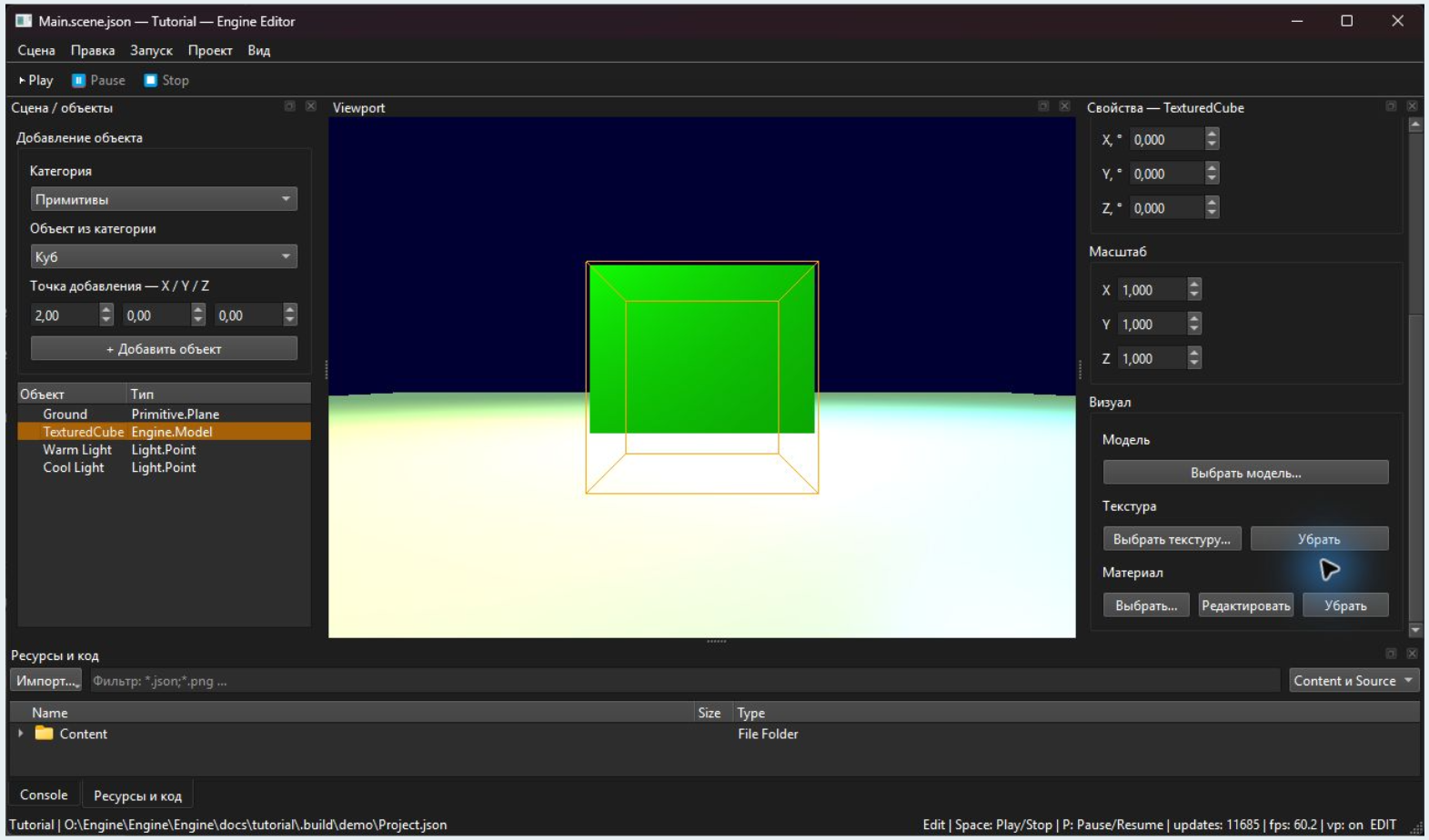
Модель, папки Materials и Textures, файл .import.json.

При добавлении

Материал назначится сам. BaseMap — текстура, Tint — базовый цвет.

Blender

Blender 4.2+. Переносится Image Texture из Base Color.



Базовый opaque-материал и UV0. Normal, roughness, metallic и сложный граф узлов не переносятся.

Редактор материала

Открыть

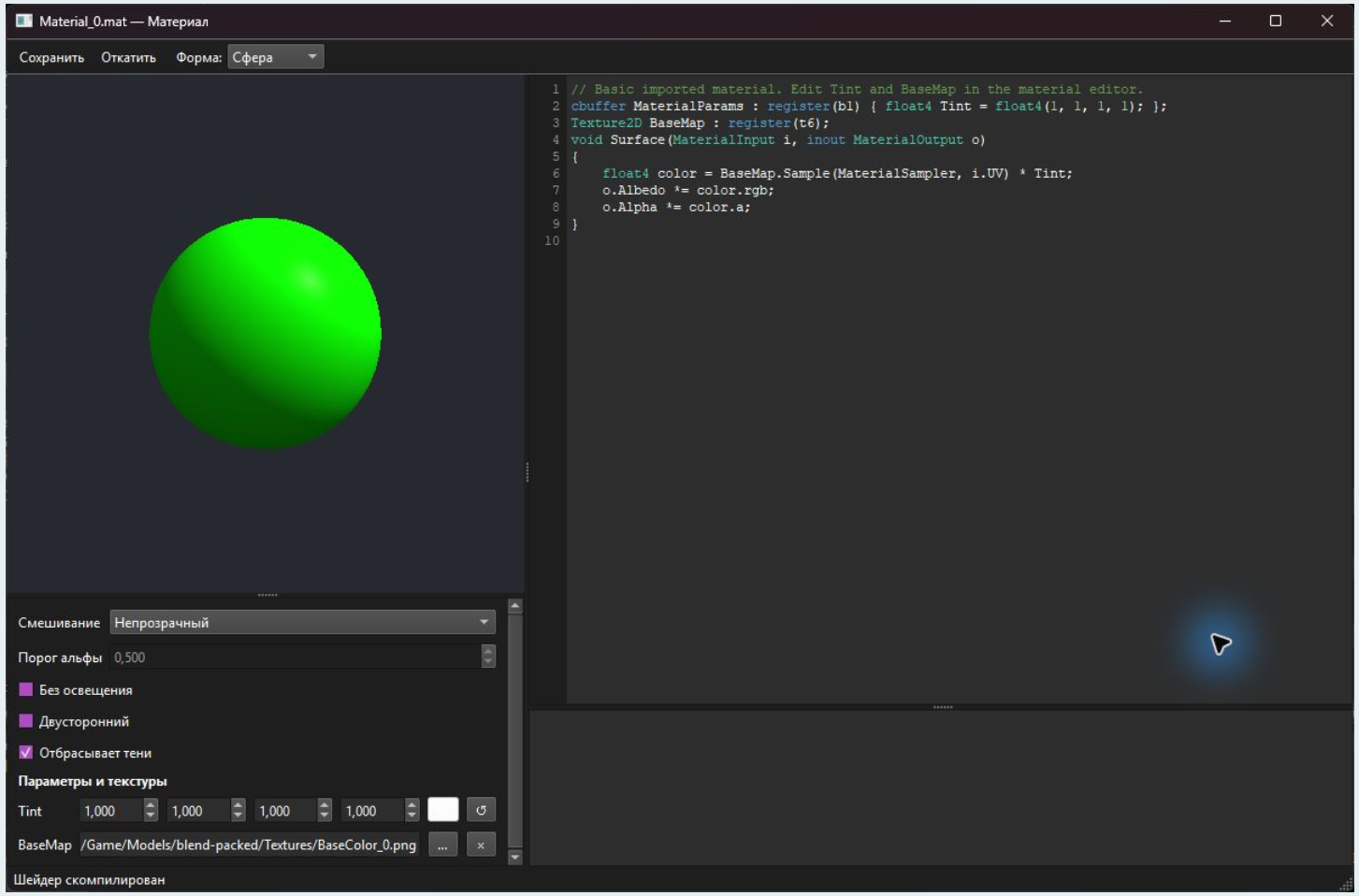
Свойства → Материал → Редактировать или двойной щелчок по .mat.

Настроить

Слева — превью, Tint и BaseMap. Справа — код HLSL поверхности.

Сохранить

Параметры видны сразу. Ctrl+S в этом окне сохраняет .mat и применяет HLSL.



Ctrl+S основного окна сохраняет сцену. F6 перечитывает материалы после внешних изменений.

Свет, частицы и небо

Свет

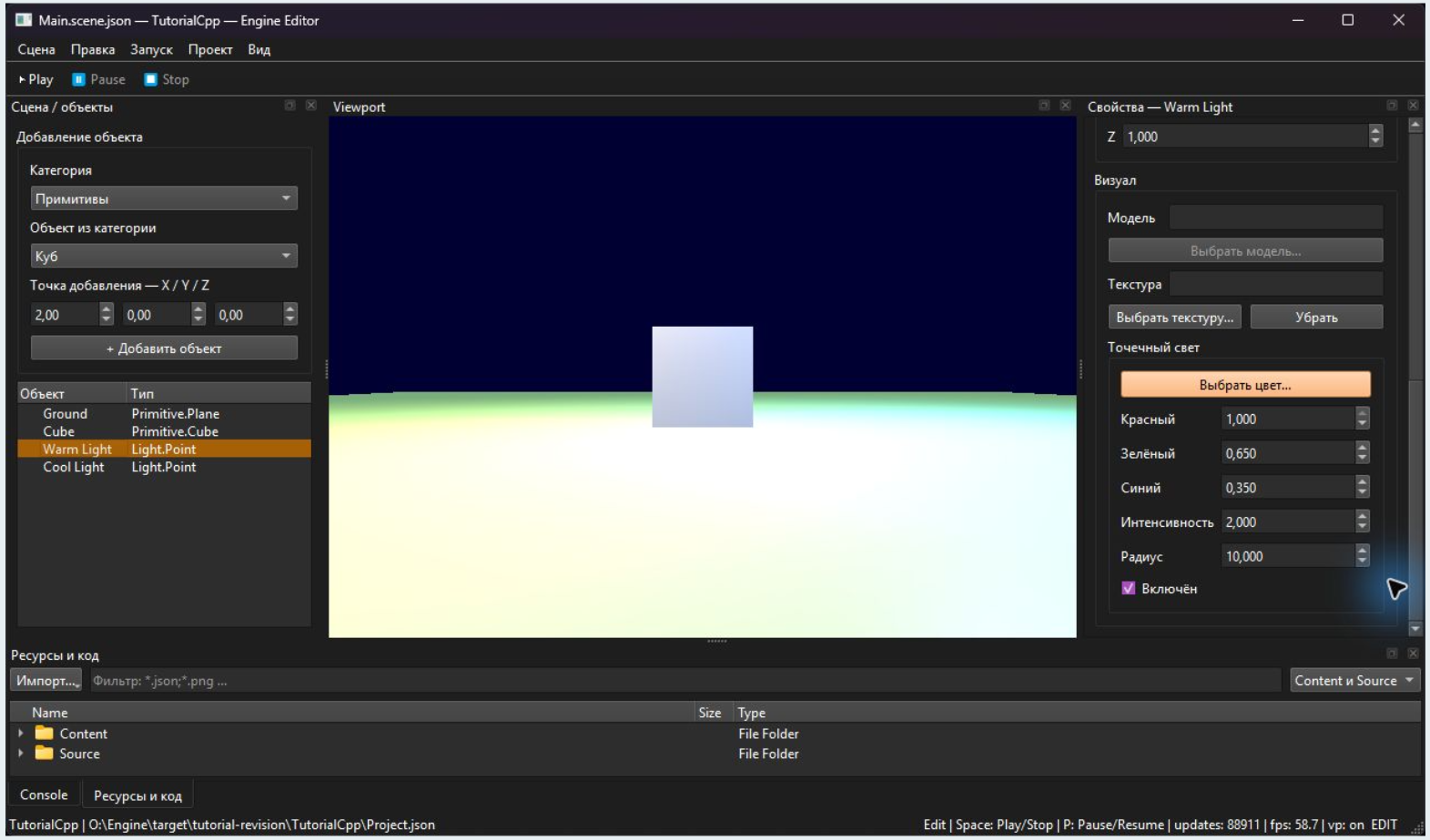
Выберите источник. В свойствах задаются цвет, интенсивность и радиус.

Частицы

В категории «Частицы» выберите искры или дым.

Небо

Категория «Окружение» → «Базовое небо».



Параметры выбранного типа находятся ниже трансформации в правой панели.

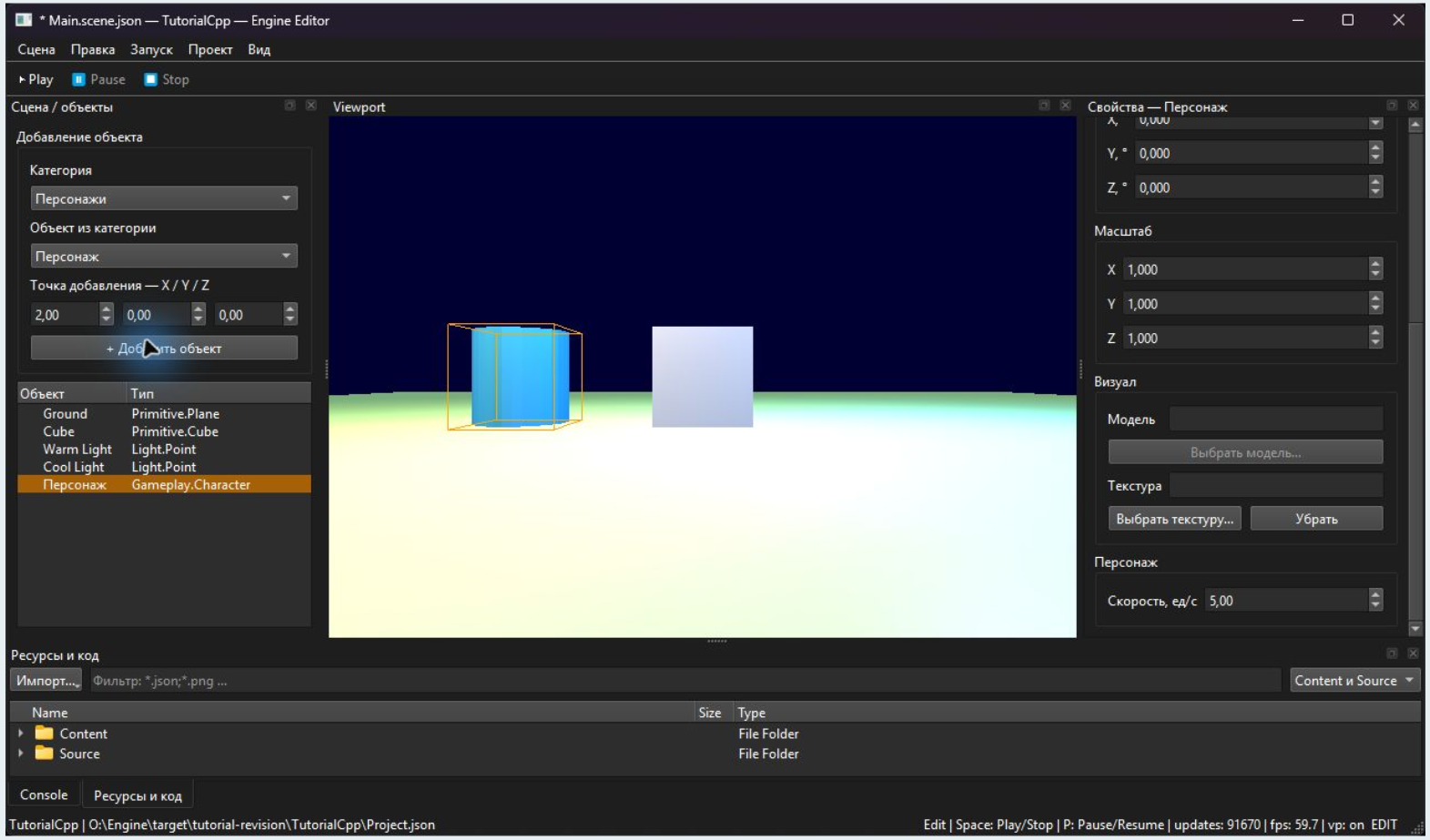
Стандартный персонаж

Добавление

Категория «Персонажи» → «Персонаж» → «Добавить объект».

Управление

Назначьте персонажа в настройках Play.



Назначение для Play

Персонаж

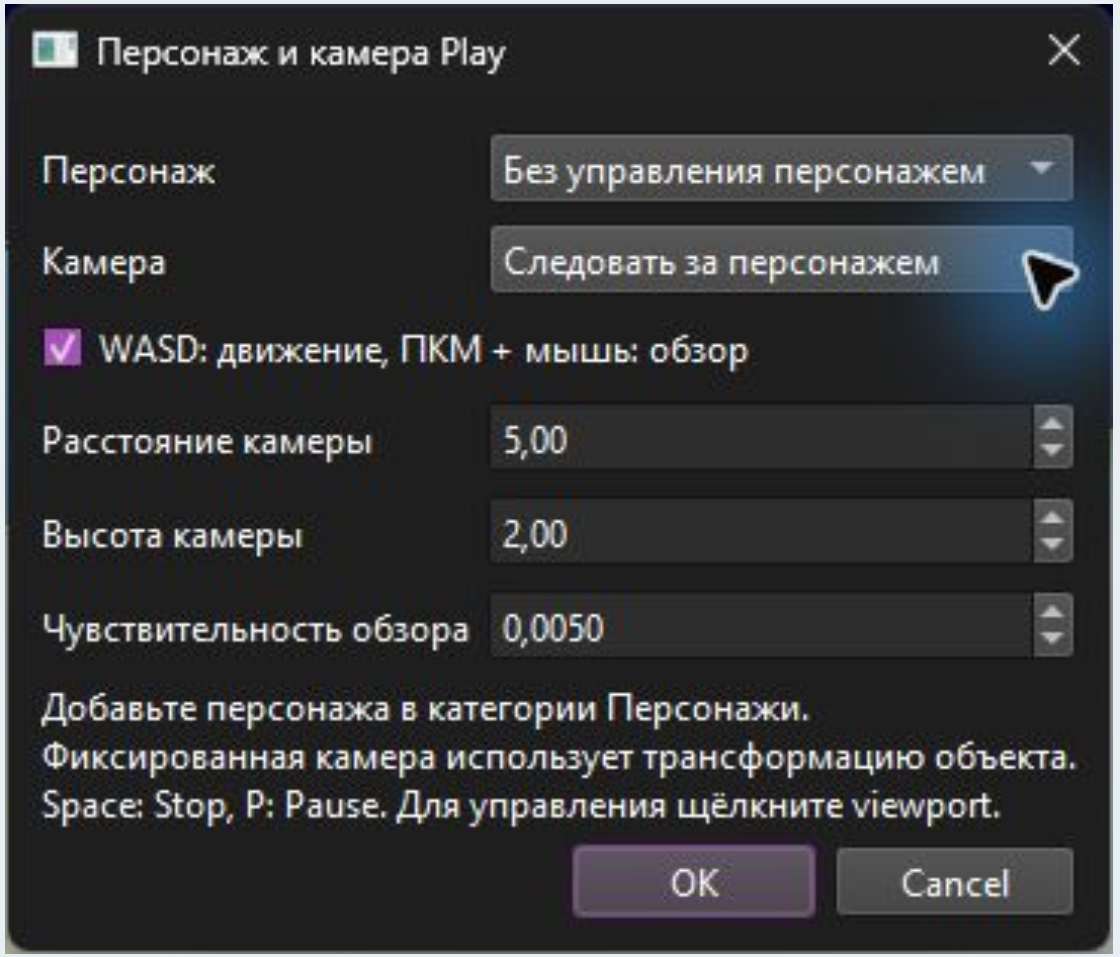
Выберите объект сцены в первом поле.

Камера

«Следовать за персонажем» или отдельная «Камера Play».

Ввод

Включите WASD и обзор с ПКМ. Затем щёлкните viewport в Play.



Запуск → Настроить Play…

Edit, Play, Pause и Stop

Play / Pause

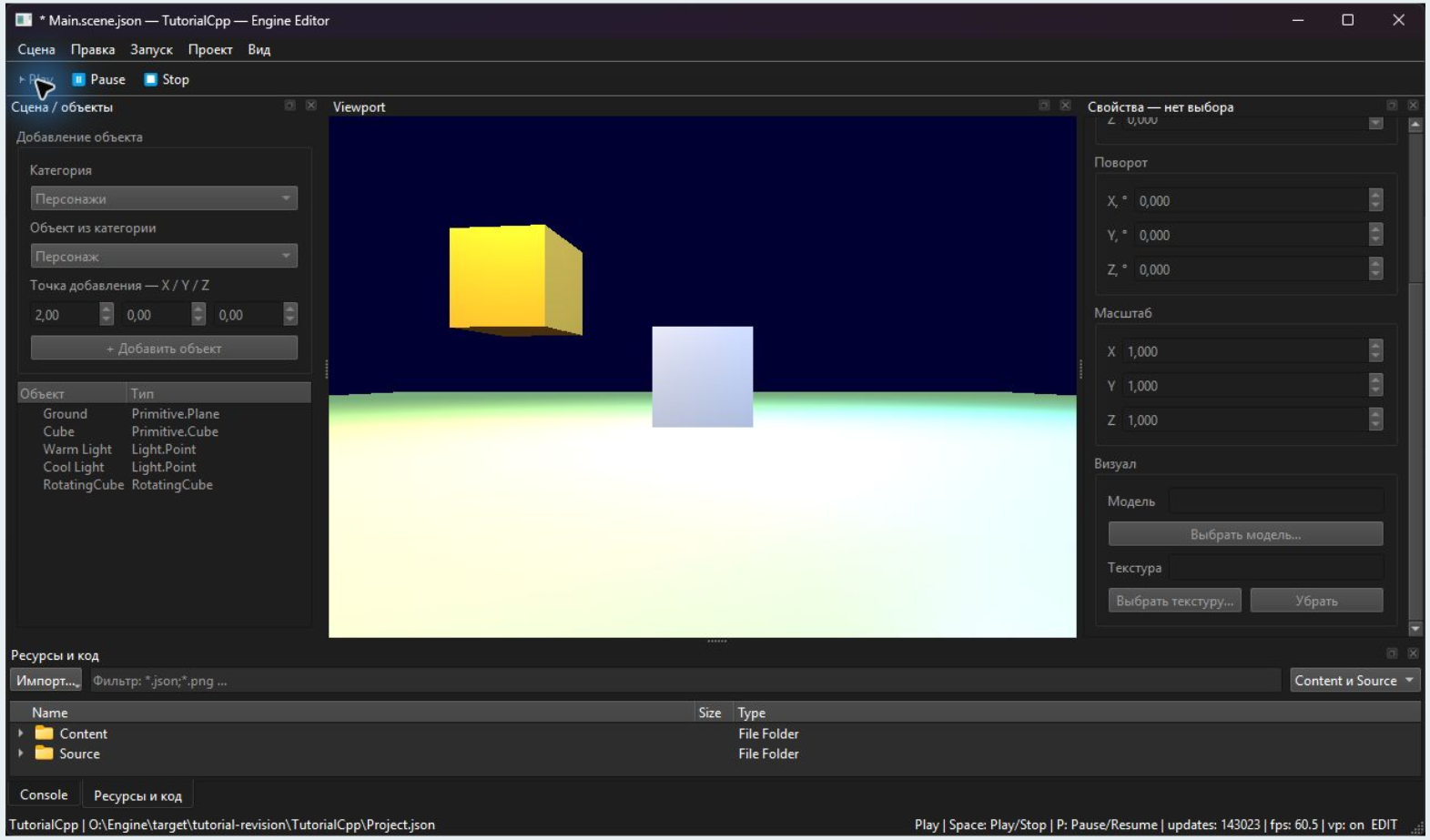
Play запускает копию сцены. Pause / Resume — пауза и продолжение.

Stop

Возвращает рабочую сцену и камеру Edit.

Отдельное окно

Отдельный запуск использует сохранённые файлы проекта.



В viewport: Space — Play / Stop, P — Pause / Resume.

Сохранение и структура проекта

Файл / папка	Назначение
Project.json	Пути к модулю, Content и стартовой сцене
Engine.lock.json	Закреплённая версия и конфигурация движка
Content/Scenes	Сцены с объектами, трансформациями и настройками Play
Content/Models, Textures, Materials	Модели, изображения, файлы .mat и .hlsl
Source и Build.ps1	Код и запуск сборки C++ проекта
Binaries	Собранные версии DLL проекта

Ctrl+S: «Сохранить и сделать стартовой». Для переноса сохраняйте всю папку проекта.

Граница между редактором и C++

Задача	Без кода	Через SDK / C++
Сцена и оформление	Модели, материалы, свет, эффекты	Свои компоненты и данные
Персонаж	Назначение для Play	Свой Character и поведение
Камера Play	Слежение или отдельная камера	Свой UpdateCamera / SceneCamera
Управление	Стандартные WASD и ПКМ	Действия, сочетания, обработчики
Каталог объектов	Встроенные типы	Регистрация своих классов

Контроллер создаётся игрой и не является отдельным добавляемым объектом сцены.

Создание C++ класса

Мастер

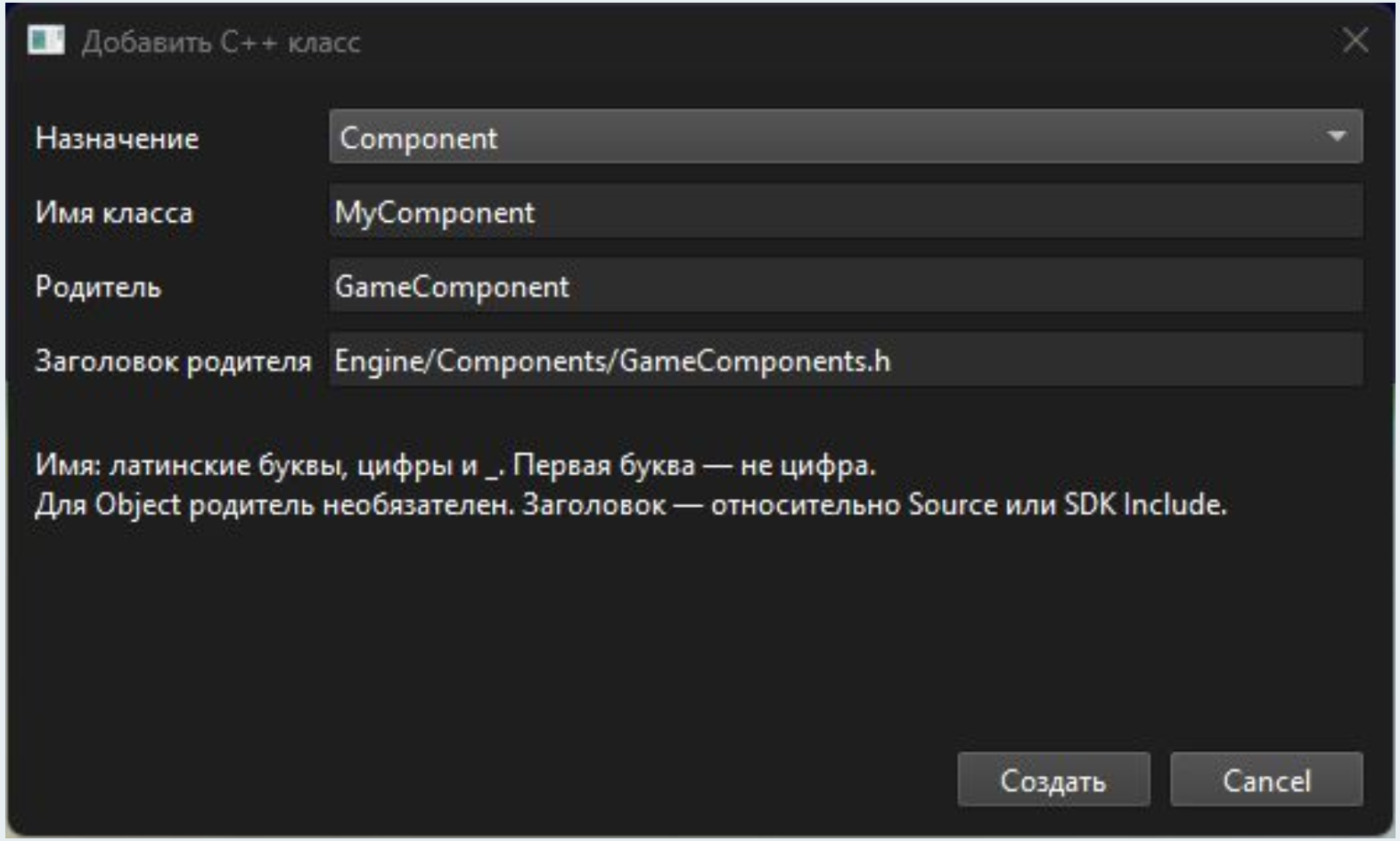
Проект → C++ → Добавить C++ класс… Выберите родителя и задайте имя.

Исходники

Мастер создаёт Source/Имя.h и Source/Имя.cpp.

Сборка

Ctrl+Alt+R — сборка и Hot Reload. Выполняйте в Edit.



Для имени класса: латиница, цифры и _. Начинайте с буквы или _.

Базовый C++ проект: вращающийся куб

Готовый пример

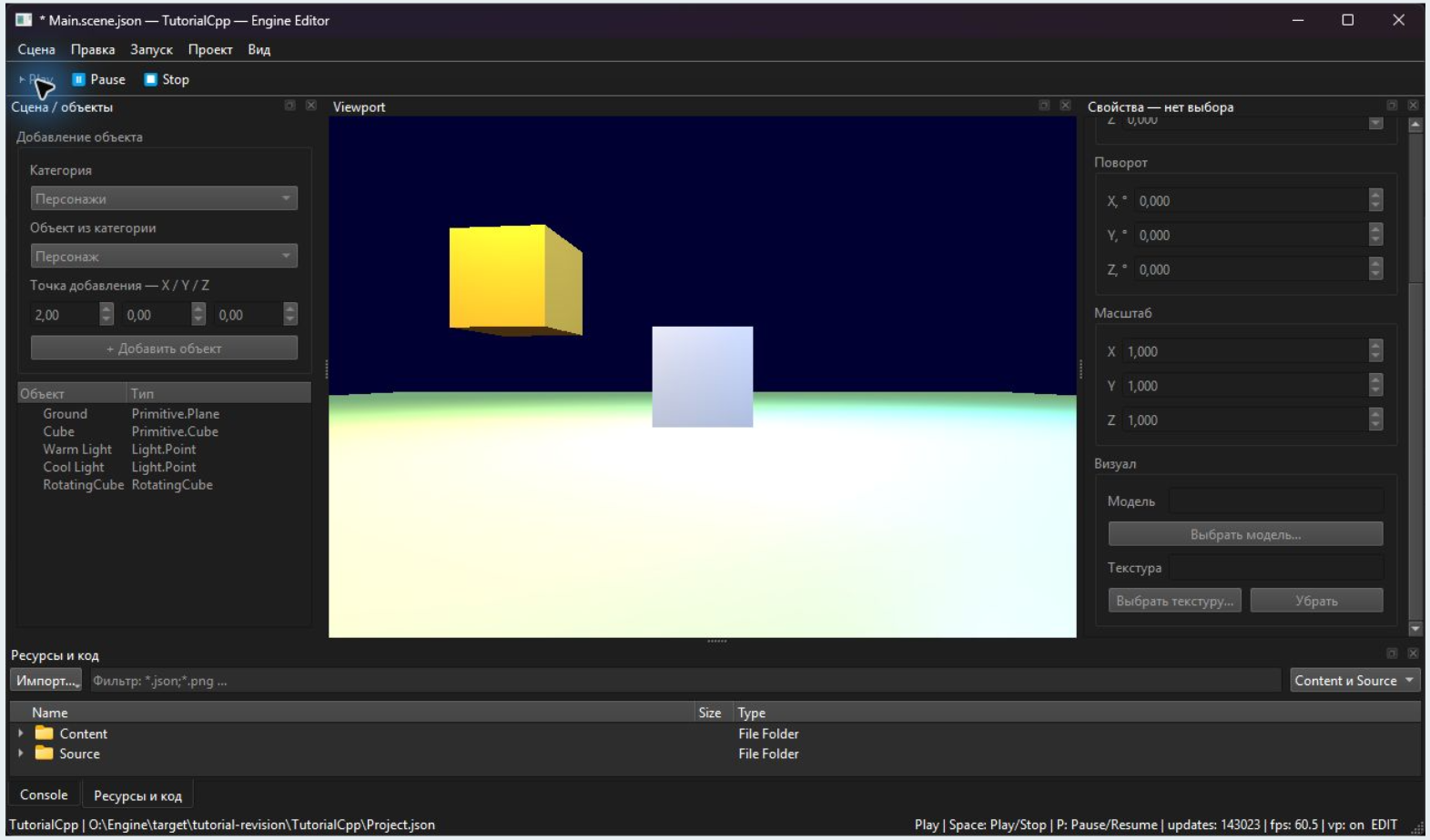
RotatingCube.h/.cpp создаются вместе с C++ проектом.

Запуск

ProjectGame добавляет куб при Play или отдельном запуске.

В Edit

Выберите «Классы проекта» → RotatingCube и добавьте объект.



Поворот вокруг Y: 90°/с. Позиция остаётся прежней.

RotatingCube: класс и геометрия

```
// Source/RotatingCube.h
#pragma once
#include <Engine/Components/ComponentAnnotations.h>
#include <Engine/Components/SpecificComponents/CubeComponent.h>

ENGINE_COMPONENT("RotatingCube")
class RotatingCube : public CubeComponent {
public:
    RotatingCube();
    void Tick(float deltaTime) override;
};

// Source/RotatingCube.cpp
#include "RotatingCube.h"
RotatingCube::RotatingCube() {
    InitCube();
    SetColor({1.0f, 0.65f, 0.2f, 1.0f});
}
```

ENGINE_COMPONENT добавляет класс в каталог проекта.

InitCube создаёт геометрию.

Конструктор задаёт исходный вид.

Вращение на месте в Tick

```
#include "RotatingCube.h"
#include <cmath>

void RotatingCube::Tick(float deltaTime)
{
    CubeComponent::Tick(deltaTime);
    auto rotation = GetRotation();
    rotation.y = std::fmod(
        rotation.y + 90.0f * deltaTime, 360.0f);
    SetRotation(rotation);
}
```

SetRotation принимает градусы.

deltaTime — время шага в секундах.

Позицию не меняем, поэтому куб вращается на месте.

В ProjectGame: SetSimulationTimeScale(1.0f). Полный оборот — четыре секунды.

Создание объекта в мире

```
// В ProjectGame::AfterInitialize()
Game::AfterInitialize();
SetSimulationTimeScale(1.0f);

auto& scene = GetScene();
std::unique_ptr<GameComponent> cube =
    std::make_unique<RotatingCube>();
cube->SetPosition({2.0f, 1.0f, 0.0f});

const auto added = scene.AddComponent(
    "MyCube", cube, "RotatingCube");
if (added.result != SceneResult::Ok &&
    added.result != SceneResult::Deferred)
    throw std::runtime_error("Cannot spawn cube");
```

Подключите RotatingCube.h и <stdexcept>.

Сцена получает владение объектом.

Имя должно быть уникальным. Во время обновления добавление может быть отложено.

Полный ProjectGame.cpp дополнительно проверяет, что куб ещё не добавлен.

Импортированная модель из C++

```
#include <Engine/Scene/SceneAssets.h>

// Файл уже импортирован через редактор.
auto model = SceneAssets::CreateModel(
    "/Game/Models/blend-packed/blend-packed.glb");
model->SetPosition({-2.0f, 0.0f, 0.0f});

const auto added = game.GetScene().AddComponent(
    "MyModel", model, SceneAssets::ModelType);
if (added.result != SceneResult::Ok &&
    added.result != SceneResult::Deferred)
    throw std::runtime_error("Cannot spawn model");
```

/Game — папка Content проекта.

CreateModel читает .import.json и назначает материал, созданный при импорте FBX/BLEND.

Подставьте путь вашей модели.

Полная функция в ModelExample.h/.cpp: SpawnModel(game, "MyModel", путь_модели).

Материал и текстура из C++

```
// object — GameComponent&, ресурс уже в Content.
const std::string material = "/Game/Materials/Wood.mat";
SceneAssets::ValidateMaterial(material);
if (!object.SetMaterialAsset(material))
    throw std::runtime_error("Material unsupported");

// Отдельная текстура объекта: материал сохраняется.
if (!object.TrySetTexture("/Game/Textures/grid.png"))
    throw std::runtime_error("Cannot set texture");

// Убрать отдельную текстуру, оставив материал:
object.TrySetTexture("");
```

BaseMap меняйте в редакторе материала.

Отдельная текстура объекта дополнительно умножает его цвет.

SetMaterialAsset назначает ссылку; шейдер компилируется при отрисовке.

Для новой геометрии используйте CreateModel. TrySetTexture меняет изображение, а не модель.

Ввод: поля и жизненный цикл

```
#pragma once
// Source/InputCube.h
#include "RotatingCube.h"
#include <Engine/InputDevice/InputActions.h>

ENGINE_COMPONENT("InputCube")
class InputCube : public RotatingCube {
public:
    void BeginPlay() override;
    void Tick(float deltaTime) override;
private:
    bool spinning_ = true;
    unsigned nextCube_ = 1;
    Input::Connection toggle_, spawn_;
    Input::Scope inputScope_;
    void SpawnCube();
};
```

BeginPlay вызывается перед первым игровым вводом и Tick.

Connection нужно хранить.

Scope объявлен последним из полей: он отключит обработчики до разрушения их состояния.

Скопируйте InputCube.h/.cpp из примеров в Source и добавьте один InputCube в сцену после сборки.

Клавиша R: остановить вращение

```
void InputCube::BeginPlay()
{
    RotatingCube::BeginPlay();
    auto& input = GetGame()->GetInput();
    input.Bind({"ToggleRotation",
        Input::Context::Game, {Keys::R}});
    toggle_ = input.Subscribe(inputScope_,
        Input::Context::Game, "ToggleRotation",
        Input::Phase::Pressed,
        [this](const Input::Event&) {
            spinning_ = !spinning_;
        });
    // Здесь же добавьте привязку N со следующего слайда.
}
```

R переключает вращение один раз на нажатие.

В Tick: RotatingCube::Tick(spinning_ ? deltaTime : 0.0f);

После Play щёлкните viewport.

Контекст Game получает ввод в сфокусированном viewport в Play; Pause и потеря фокуса сбрасывают действия.

Клавиша N: создать новый куб

```
// Продолжение InputCube::BeginPlay()
input.Bind({"SpawnCube",
    Input::Context::Game, {Keys::N}});
spawn_ = input.Subscribe(inputScope_,
    Input::Context::Game, "SpawnCube",
    Input::Phase::Pressed,
    [this](const Input::Event&) { SpawnCube(); });

// В InputCube::SpawnCube()
std::unique_ptr<GameComponent> cube =
    std::make_unique<RotatingCube>();
cube->SetPosition(GetLocalPosition() +
    glm::vec3(2.0f * number, 0, 0));
const auto added = scene.AddComponent(
    name, cube, "RotatingCube");
```

SpawnCube получает GetGame()->GetScene().

number — счётчик, name — свободное имя SpawnedCubeN.

В полном файле проверяются имя и результат добавления.

Новый куб появляется в копии Play. После Stop он исчезает вместе с этой копией.

Сочетания и переназначение

```
// В BeginPlay после привязки ToggleRotation
input.Unbind(Input::Context::Game, "ToggleRotation");
input.Bind({"ToggleRotation", Input::Context::Game,
            {Keys::Control, Keys::R}});

// Две привязки одной оси
input.Bind({"MoveX", Input::Context::Game,
            {Keys::D}, 1});
input.Bind({"MoveX", Input::Context::Game,
            {Keys::A}, -1});
```

Обработчик ToggleRotation остаётся прежним.

Ctrl + R — одновременное удержание клавиш.

Для MoveX подпишитесь на Phase::Axis и читайте e.value. A + D даёт ноль.

Своя схема управления

```
class MyController : public CharacterController {
public:
    using CharacterController::CharacterController;
    void BindInput() override {
        game_.GetInput().Bind({"Go",
            Input::Context::Game, {Keys::I}});
        connections_.push_back(game_.GetInput().Subscribe(
            inputScope_, Input::Context::Game, "Go",
            Input::Phase::Axis,
            [this](const Input::Event& e) {
                forward_ = e.value;
            }));
    }
};
```

I задаёт движение вперёд и заменяет стандартные привязки.

Game выбирает контроллер в методе фабрики. Пример — в README.txt.

Для своей камеры переопределите UpdateCamera().

Выберите MyGame: Проект → C++ → Типы C++ проекта…; персонажа назначьте в настройках Play.

Обновление движка и проекта

Версия

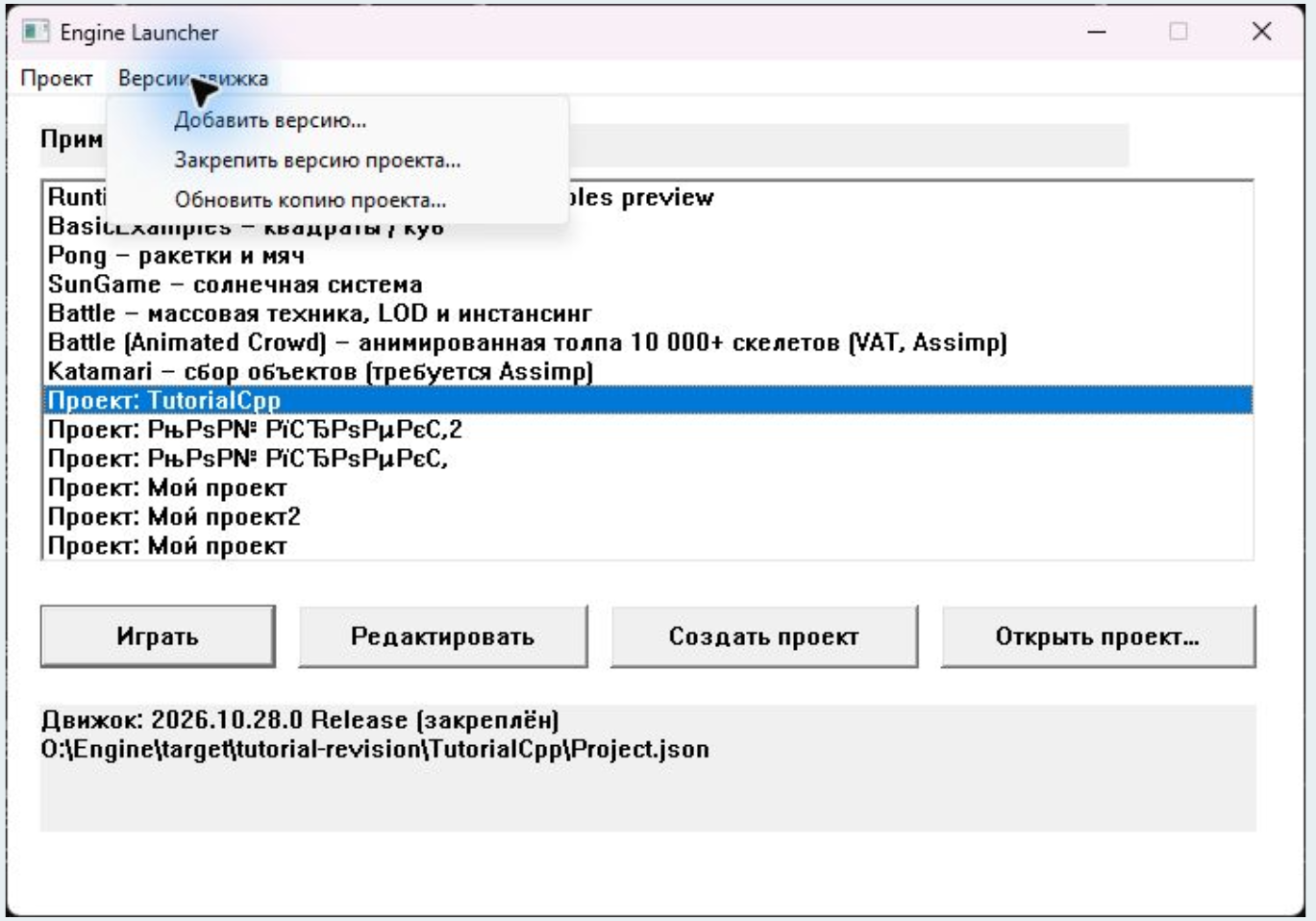
Engine.lock.json закрепляет выпуск и конфигурацию.

Обновление

Добавьте выпуск. Затем выберите «Обновить копию проекта…».

Проект C++

Пересоберите DLL новым SDK и проверьте сцену и Play.



Исходный проект сохраняется; обновление выполняется в копии.

Частые вопросы при первом запуске

Симптом	Что проверить
Персонаж не двигается	Play, фокус viewport, назначение и флажок управления
BLEND не импортируется	Blender 4.2+, доступные картинки или упакованные текстуры
Материал рисуется шахматкой	Ошибки HLSL, путь .mat; после исправления нажмите F6
Новый класс не виден	Успешная сборка, регистрация и Hot Reload в Edit
Ошибка сборки / загрузки	Console, CodeBuild.log и логи кандидата в Binaries
Пропала модель или текстура	Ресурс импортирован в Content и перенесён вместе с проектом